

# PHÂN TÍCH THỐNG KÊ NHIỀU CHIỀU (MAT3452)

## Bài tập tuần 2: Phân phối chuẩn một chiều

Trần Đăng Quang Minh - MSV: 23000141

**Câu 1.** Tập dữ liệu **hills** trong thư viện MASS gồm thời gian kỷ lục của 35 cuộc đua trên các ngọn đồi ở Scotland vào năm 1984. Với mỗi cuộc đua trên từng ngọn đồi, bộ dữ liệu cung cấp độ dài đường đua (dặm), độ cao đạt được (feet) và thời gian chạy của người dành chiến thắng (phút).

(i)

```
library(MASS)
data(hills)

head(hills)

##           dist climb  time
## Greenmantle  2.5   650 16.083
## Carnethy     6.0  2500 48.350
## Craig Dunain  6.0   900 33.650
## Ben Rha      7.5   800 45.600
## Ben Lomond   8.0  3070 62.267
## Goatfell    8.0  2866 73.217

colMeans(hills)

##           dist           climb           time
##  7.528571 1815.314286   57.875714

sapply(colnames(hills), function(x) {
  print(sd(hills[, x]))
})

## [1] 5.523936
## [1] 1619.151
## [1] 50.04072

##           dist           climb           time
##  5.523936 1619.150536   50.040716
```

(ii)

```
sapply(colnames(hills), function(x) {
  st <- shapiro.test(hills[[x]])
  print(st)
  if (st$p.value >= 0.05) {
    print("Là phân phối chuẩn")
  } else {
    print("Không phải phân phối chuẩn")
  }
})

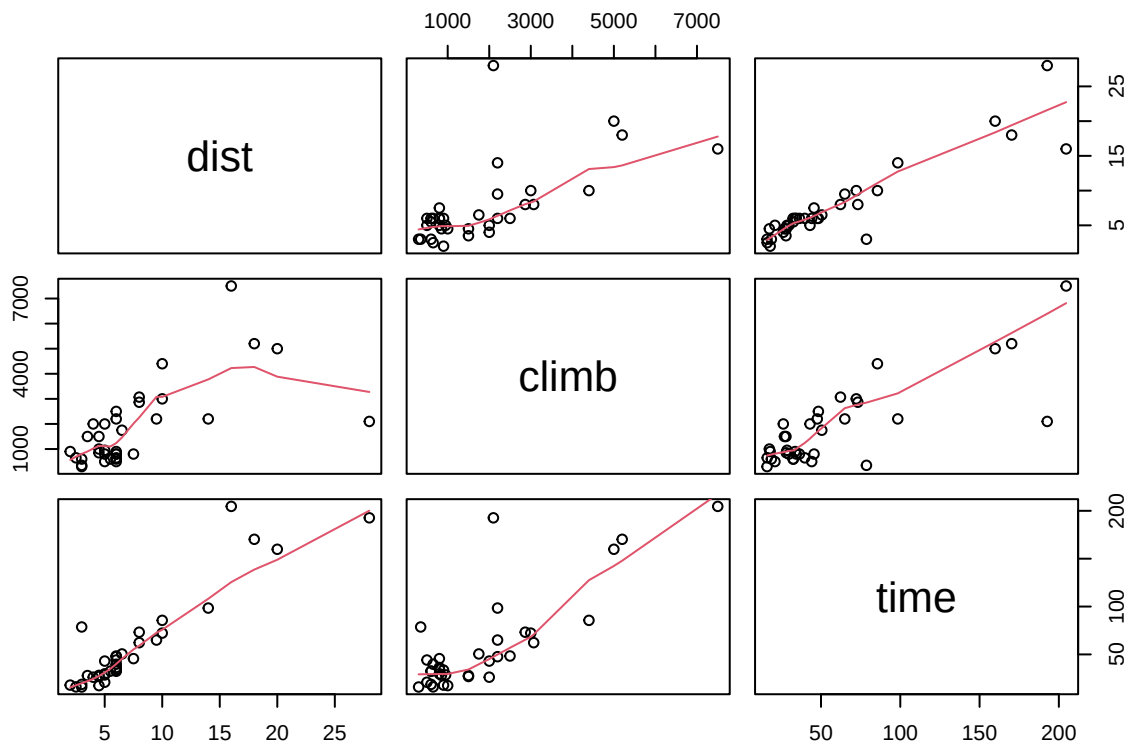
##
## Shapiro-Wilk normality test
##
## data: hills[[x]]
## W = 0.74904, p-value = 2.362e-06
##
## [1] "Không phải phân phối chuẩn"
```

```
##
## Shapiro-Wilk normality test
##
## data: hills[[x]]
## W = 0.79245, p-value = 1.456e-05
##
## [1] "Không phải phân phối chuẩn"
##
## Shapiro-Wilk normality test
##
## data: hills[[x]]
## W = 0.73673, p-value = 1.458e-06
##
## [1] "Không phải phân phối chuẩn"

##                                dist                                climb
## "Không phải phân phối chuẩn" "Không phải phân phối chuẩn"
##                                time
## "Không phải phân phối chuẩn"
```

(iii)

```
pairs(hills, panel = panel.smooth)
```



**Câu 2.** Tập dữ liệu **iris** trong thư viện **datasets** gồm một số đặc điểm của các bông hoa diên vĩ. Với mỗi một bông hoa, bộ dữ liệu cung cấp độ dài, độ rộng của cánh hoa và đài hoa (cm) và loài.

(i)

```
library(datasets)
data(iris)
head(iris)

##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5         1.4         0.2   setosa
## 2          4.9         3.0         1.4         0.2   setosa
## 3          4.7         3.2         1.3         0.2   setosa
## 4          4.6         3.1         1.5         0.2   setosa
## 5          5.0         3.6         1.4         0.2   setosa
## 6          5.4         3.9         1.7         0.4   setosa
```

```
df <- iris[1:4]
colMeans(df)

## Sepal.Length Sepal.Width Petal.Length Petal.Width
##      5.843333      3.057333      3.758000      1.199333

sapply(colnames(df), function(x) {
  sd(df[, x])
})
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##      0.8280661      0.4358663      1.7652982      0.7622377
```

(ii)

```
sapply(colnames(df), function(x) {
  st <- shapiro.test(df[[x]])
  print(st)
  if (st$p.value >= 0.05) {
    print("Là phân phối chuẩn")
  } else {
    print("Không phải phân phối chuẩn")
  }
})
```

```
##
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.97609, p-value = 0.01018
##
## [1] "Không phải phân phối chuẩn"
##
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.98492, p-value = 0.1012
##
## [1] "Là phân phối chuẩn"
##
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.87627, p-value = 7.412e-10
##
## [1] "Không phải phân phối chuẩn"
##
```

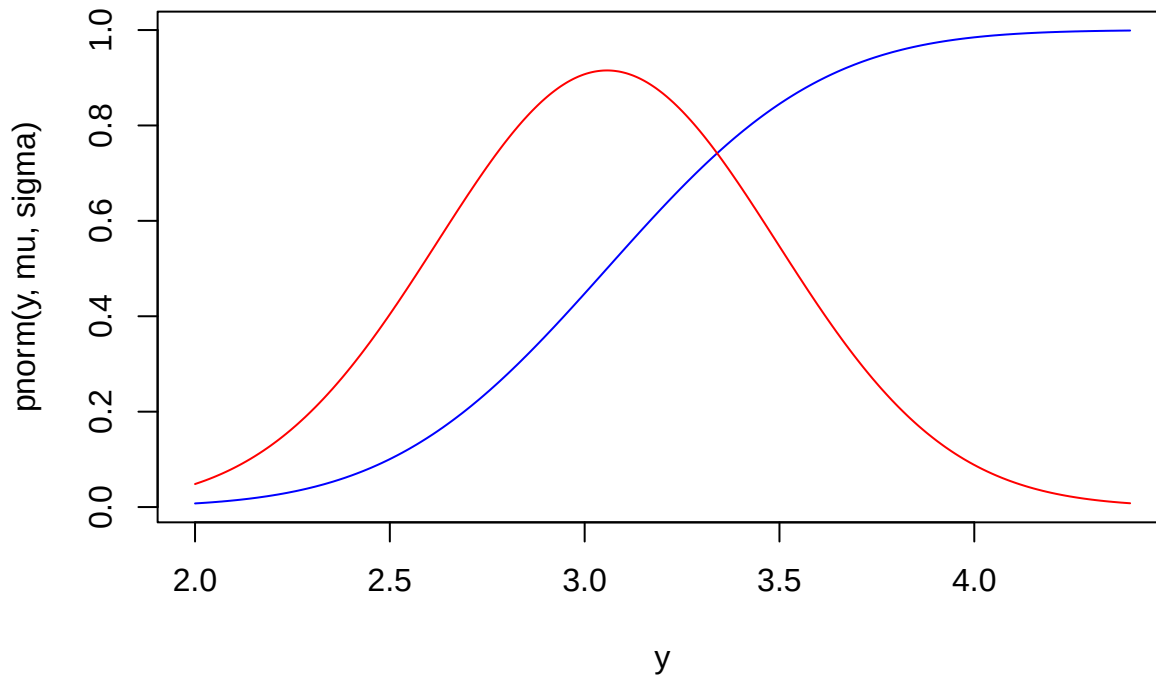
```
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.90183, p-value = 1.68e-08
##
## [1] "Không phải phân phối chuẩn"

##                Sepal.Length                Sepal.Width
## "Không phải phân phối chuẩn"        "Là phân phối chuẩn"
##                Petal.Length                Petal.Width
## "Không phải phân phối chuẩn" "Không phải phân phối chuẩn"

(iii)
sapply(colnames(df), function(x) {
  st <- shapiro.test(df[[x]])
  mu <- mean(df[[x]])
  sigma <- sd(df[[x]])
  print(st)
  y <- seq(min(df[[x]]), max(df[[x]]), length = 1000)
  if (st$p.value >= 0.05) {
    plot(y, pnorm(y, mu, sigma), type = "l", main = "Hàm mật độ và hàm phân phối tích lũy", col = "blue")
    lines(y, dnorm(y, mu, sigma), main = "", xlab = "", ylab = "", col = "red")
  }
})

##
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.97609, p-value = 0.01018
##
##
## Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.98492, p-value = 0.1012
```

## Ham mat do va ham phan phoi tich luy



```
##
##  Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.87627, p-value = 7.412e-10
##
##
##  Shapiro-Wilk normality test
##
## data:  df[[x]]
## W = 0.90183, p-value = 1.68e-08
##
## $Sepal.Length
## NULL
##
## $Sepal.Width
## NULL
##
## $Petal.Length
## NULL
##
## $Petal.Width
## NULL
```

(iv)

```
t.test(df[, "Sepal.Width"], mu = 3.5, alternative = "less", conf.level = 0.95)
```

```
##
##  One Sample t-test
```

```
##
## data: df[, "Sepal.Width"]
## t = -12.439, df = 149, p-value < 2.2e-16
## alternative hypothesis: true mean is less than 3.5
## 95 percent confidence interval:
##      -Inf 3.116237
## sample estimates:
## mean of x
## 3.057333

(v)
a <- qnorm(0.7, mean = mean(iris$Sepal.Width), sd = sd(iris$Sepal.Width))
a
```

```
## [1] 3.285902
```

```
(vi)
setosa <- subset(iris, iris$Species == "setosa")
versicolor <- subset(iris, iris$Species == "versicolor")
# H0: Độ rộng TB bằng nhau, H1: Độ rộng TB khác nhau
t.test(setosa$Sepal.Width, versicolor$Sepal.Width, conf.level = 0.99)
```

```
##
## Welch Two Sample t-test
##
## data: setosa$Sepal.Width and versicolor$Sepal.Width
## t = 9.455, df = 94.698, p-value = 2.484e-15
## alternative hypothesis: true difference in means is not equal to 0
## 99 percent confidence interval:
## 0.4750576 0.8409424
## sample estimates:
## mean of x mean of y
## 3.428 2.770
```

**Câu 3.** Sinh 30 mẫu ngẫu nhiên, mỗi mẫu gồm 20 quan sát có phân phối chuẩn với kỳ vọng 158 và phương sai 8.

```
(i)
mu <- 158
sigma <- sqrt(8)
data <- as.data.frame(matrix(rnorm(30 * 20, mu, sigma), nrow = 20, ncol = 30))
head(data)
```

```
##      V1      V2      V3      V4      V5      V6      V7      V8
## 1 150.7458 155.6152 157.0196 156.4890 157.0053 159.7894 160.7423 154.8793
## 2 157.8706 160.3613 160.4827 152.8943 157.2416 163.6365 158.9862 156.1340
## 3 154.3678 155.0610 158.5353 159.0110 160.5348 156.2704 157.6852 157.2934
## 4 158.3728 159.7466 156.8098 159.1635 160.2113 156.5719 156.5226 152.3420
## 5 158.2889 160.6751 156.2302 158.5519 156.5496 158.8049 157.1333 159.7443
## 6 161.6959 158.4585 160.9395 159.1774 158.4666 158.4571 155.0913 160.0934
##      V9      V10     V11     V12     V13     V14     V15     V16
## 1 157.7380 159.8745 153.9034 159.2186 157.7366 160.7173 160.1762 155.2519
## 2 160.3692 157.5076 162.2935 159.8157 152.5963 156.5545 158.7945 160.7084
## 3 159.8430 151.8468 160.9507 157.8875 154.9204 156.8714 155.6249 161.2239
## 4 152.7135 159.6684 159.4019 158.8742 152.4055 161.4288 159.6092 154.3051
## 5 157.8597 160.4263 159.1694 158.9993 157.1343 157.2849 156.2686 157.5257
## 6 159.9402 157.7896 156.1560 162.2555 155.4079 163.7548 161.6999 155.0213
##      V17     V18     V19     V20     V21     V22     V23     V24
```

```
## 1 153.8516 161.1770 158.3280 160.4596 158.7722 155.0664 162.2710 159.6811
## 2 160.6738 159.4530 157.2724 156.2443 160.9361 161.4886 157.5135 155.6862
## 3 152.1141 163.1024 155.9845 153.4593 158.3697 159.1285 157.3980 157.2913
## 4 157.2642 159.3534 158.3635 163.1847 156.1700 160.0272 159.0937 153.9109
## 5 160.0507 156.7399 159.7590 154.3734 156.7919 159.3137 158.0102 155.3359
## 6 159.8863 152.9181 161.3309 159.6941 158.0839 163.0787 160.3494 155.4435
##      V25      V26      V27      V28      V29      V30
## 1 159.5525 159.1229 160.0271 158.4661 157.5609 160.2388
## 2 157.4515 158.3765 157.5604 154.2124 156.5186 158.5325
## 3 156.9772 157.2245 156.2948 158.1092 154.0481 160.1170
## 4 156.6676 159.8572 155.2269 154.5426 154.2980 158.7451
## 5 159.9662 152.1764 154.2551 151.7264 155.9993 155.1819
## 6 157.8844 161.6563 152.8369 162.0998 160.7962 159.1009
```

```
sapply(colnames(data), function(x) {
  xx <- data[[x]]
  xbar <- mean(xx)
  n <- length(xx)
  z <- qnorm(0.99)
  val <- z * sigma / sqrt(n)
  ci <- c(xbar - val, xbar + val)
  ci
  # library(BSDA)
  # z.test(data[[x]], y = NULL, mu, sigma.x = sigma, alternative = "two.sided", sigma.y = NULL, conf.level
})
```

```
##      V1      V2      V3      V4      V5      V6      V7      V8
## [1,] 156.8049 157.1954 156.2146 156.0966 156.4769 156.6658 156.3497 156.4573
## [2,] 159.7475 160.1381 159.1573 159.0392 159.4195 159.6084 159.2923 159.4000
##      V9      V10     V11     V12     V13     V14     V15     V16
## [1,] 157.3996 155.9330 156.3031 157.4759 155.2130 156.8664 157.3415 155.1703
## [2,] 160.3423 158.8756 159.2457 160.4185 158.1557 159.8090 160.2842 158.1129
##      V17     V18     V19     V20     V21     V22     V23     V24
## [1,] 155.7821 157.3742 156.9133 155.9080 155.9137 157.5181 156.2255 155.9790
## [2,] 158.7247 160.3168 159.8559 158.8507 158.8564 160.4607 159.1681 158.9216
##      V25     V26     V27     V28     V29     V30
## [1,] 157.0674 156.8965 154.3769 155.6824 155.8912 156.3539
## [2,] 160.0100 159.8391 157.3196 158.6250 158.8338 159.2965
```

(ii)

```
sapply(colnames(data), function(x) {
  xx <- data[[x]]
  xbar <- mean(xx)
  n <- length(xx)
  z <- qnorm(0.99)
  val <- z * sigma / sqrt(n)
  ci <- c(xbar - val, xbar + val)
  ci[2] - ci[1]
  # ci <- z.test(data[[x]], y = NULL, mu, sigma.x = sigma, alternative = "two.sided", sigma.y = NULL, conf
  # ci[2] - ci[1]
})
```

```
##      V1      V2      V3      V4      V5      V6      V7      V8
## 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623
##      V9      V10     V11     V12     V13     V14     V15     V16
## 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623
##      V17     V18     V19     V20     V21     V22     V23     V24
## 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623
```

```
##      V25      V26      V27      V28      V29      V30
## 2.942623 2.942623 2.942623 2.942623 2.942623 2.942623
```

(iii)

```
sapply(colnames(data), function(x) {
  xx <- data[[x]]
  xbar <- mean(xx)
})
```

```
##      V1      V2      V3      V4      V5      V6      V7      V8
## 158.2762 158.6667 157.6859 157.5679 157.9482 158.1371 157.8210 157.9286
##      V9      V10     V11     V12     V13     V14     V15     V16
## 158.8709 157.4043 157.7744 158.9472 156.6844 158.3377 158.8128 156.6416
##      V17     V18     V19     V20     V21     V22     V23     V24
## 157.2534 158.8455 158.3846 157.3794 157.3850 158.9894 157.6968 157.4503
##      V25     V26     V27     V28     V29     V30
## 158.5387 158.3678 155.8483 157.1537 157.3625 157.8252
```

```
print(paste0("Mẫu thứ ", which.max(colMeans(data)), " có GTTB lớn nhất"))
```

```
## [1] "Mẫu thứ 22 có GTTB lớn nhất"
```

```
max(colMeans(data))
```

```
## [1] 158.9894
```